ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Departamento de Ciencias de la Computación

Desarrollo de VideoJuegos

Patrones de Diseño

Que son?

Los patrones de diseño son soluciones probadas y reutilizables para resolver problemas comunes en la programación y el diseño de software.

En el desarrollo de videojuegos tienen una aplicación muy amplia. Algunos patrones específicos ayudan a estructurar el código, gestionar el flujo de trabajo, mejorar la eficiencia y crear interacciones coherentes en el juego

Porqué son importantes?

- Permiten que el código sea más legible y fácil de mantener
- Hacen posible que el equipo de desarrollo responda rápidamente a cambios en las mecánicas del juego, escalando o modificando funcionalidades.
- Abordan problemas comunes en el desarrollo de videojuegos para lograr un código modular y escalable,
- Permiten optimizar tanto el rendimiento del juego como la eficiencia en el proceso de desarrollo.

Patrones de Diseño

Patrón	Propósito	Uso	Ejemplo
Singleton	Asegura que una clase solo tenga una instancia y proporciona un punto de acceso global a ella.	Administrar recursos globales y únicos en el juego, como el administrador de sonido, el sistema de puntuación, o el estado de partida. Al asegurar que solo haya una instancia, se evita el uso excesivo de memoria o conflictos entre múltiples instancias.	En un juego, se puede tener un solo GameManager para manejar las reglas generales y el estado de juego.
Factory	Proporciona una interfaz para crear objetos en una superclase, permitiendo que las subclases decidan qué clase instanciar	Se utiliza para crear distintos tipos de objetos según las necesidades de juego, como enemigos, ítems o personajes, especialmente si tienen variaciones y necesitan crearse en diferentes momentos.	Una fábrica de enemigos puede generar diferentes tipos de enemigos (zombis, alienígenas, robots) dependiendo del nivel.
Observer	Define una dependencia de uno a muchos entre objetos de manera que cuando un objeto cambia su estado, todos sus dependientes son notificados.	Ideal para sistemas de notificaciones. Se puede usar para actualizar la interfaz cuando el jugador gana puntos o para manejar eventos en un juego multijugador en el que varios objetos (o jugadores) están observando un estado central.	En un juego de estrategia, si una base es destruida, los aliados que dependen de ella pueden reaccionar inmediatamente.

Patrones de Diseño

Patrón	Propósito	Uso	Ejemplo
State	Permite que un objeto altere su comportamiento cuando cambia su estado.	Muy útil en la lógica de IA de los personajes, como un enemigo que cambia de comportamiento entre estados como “patrullando”, “alerta” y “atacando” según la situación.	Un enemigo en un juego de sigilo puede cambiar de estado a “persiguiendo” al jugador al detectar movimiento.
Strategy	Permite definir una familia de algoritmos, encapsulándolos y haciéndolos intercambiables para que puedan ser seleccionados y usados en tiempo de ejecución.	Se usa para manejar diferentes estrategias de IA o de movimiento, por ejemplo, para cambiar la estrategia de ataque de un enemigo dependiendo de la situación.	En un juego de lucha, un personaje puede cambiar entre estrategias de ataque y defensa.
Command	Encapsula una solicitud como un objeto, permitiendo parametrizar clientes con diferentes solicitudes, hacer un seguimiento de comandos ejecutados y soportar deshacer.	Es útil para implementar acciones como movimientos, ataques o habilidades especiales de los personajes, especialmente si estas pueden deshacerse o repetirse.	En un juego de estrategia, el patrón de comando permite realizar y deshacer órdenes, como moverse o construir, permitiendo al jugador cancelar acciones.
Prototype	Permite la creación de nuevos objetos copiando o clonando una instancia inicial en lugar de instanciar nuevos objetos.	Facilita la creación de múltiples instancias de un objeto con ciertas características, como enemigos o ítems, sin necesidad de recalcular toda la configuración.	Un juego de disparos puede usar un prototipo para generar nuevas balas basadas en un modelo inicial.

Motores de Juegos

Que son?

Un framework de software y entorno de desarrollo que proporciona herramientas esenciales —como motor gráfico 2D/3D, físicas, sonido e IA— para crear videojuegos sin programar todo desde cero.

Los motores de videojuegos son realmente el núcleo en el que se unen todas las piezas que integran un videojuego, desde las más artísticas o creativas a la programación.

Los motores de videojuegos responden a un modelo de software muy concreto.

Motores de videojuegos o *game engines*

- Herramientas gracias a las cuales pueden diseñar escenarios, animar personajes, añadir sonidos….
- Están diseñados principalmente para hacer videojuegos, pero actualmente la mayoría de ellos permiten crear video, animaciones o aplicaciones de todo tipo y para múltiples plataformas.
- Son plataformas en las que el desarrollador se encuentra con una serie de librerías de programación con las que trabajar en el diseño y la creación de un juego.
- Cuenta con una **estructura sobre la que empezar a trabajar aplicando sus propias ideas**, pero contando con el soporte necesario para llevar a cabo todas esas cuestiones implicadas en este proceso.

Componentes

Motor gráfico o de renderizado. De él depende cómo se muestran las imágenes en la pantalla (2D, 3D) pero también todo lo relacionado con las texturas y la iluminación.

Motor de sonidos. Tiene tanta importancia como el motor gráfico y es el encargado de sincronizar el sonido, cargar las diferentes pistas o modificarlas a su gusto.

Física. Simula las leyes físicas en el juego, como colisiones, gravedad, inercia y movimiento de objetos. Contribuye a la simulación de entornos realistas.

Motor de entrada. Maneja la interacción del usuario con el juego, ya sea mediante teclado, ratón, controladores de juego, pantalla táctil u otros dispositivos de entrada.

Motor de red. En juegos multijugador, gestiona la comunicación y sincronización entre diferentes instancias del juego a través de la red.

Scripting. Se centra en el funcionamiento tanto de los personajes del juego como del resto de objetos que aparecen. El programador debe contar con la opción de poder crear sus propios *scripts* o modificar los del motor, para poder aplicar todas las ideas que tiene.

Componentes

Inteligencia artificial. Proporciona herramientas y algoritmos para la implementación de la inteligencia artificial en el juego, incluyendo comportamientos de NPCs y toma de decisiones.

Motor de escenas. Organiza y gestiona las diferentes escenas del juego, que pueden incluir niveles, menús, cinemáticas y otras secciones distintas.

Motor de animación. Controla la animación de personajes y objetos en el juego, incluyendo la transición entre diferentes estados de animación.

Motor de Renderización de Partículas. Manipula la renderización de efectos de partículas, como humo, fuego, chispas y otros elementos visuales dinámicos.

Motor de colisiones. Detecta y resuelve colisiones entre objetos en el juego. Contribuye a la simulación de interacciones físicas y la respuesta a eventos de colisión.

Motor de persistencia. Gestiona la capacidad de guardar y cargar partidas. Esto implica guardar y recuperar el estado actual del juego.

Componentes

Herramientas de Desarrollo. Incluyen un conjunto de herramientas para diseñar, construir y depurar el juego. Pueden incluir editores de escenas, herramientas de diseño de niveles, y otros recursos para facilitar el desarrollo.

Principales motores

A la hora de elegir entre un motor u otro **hay que tener en cuenta toda una serie de factores:**

- Presupuesto disponible,
- Equipo material con el que se va a trabajar,
- Experiencia del equipo
- Plazos establecidos a cumplir.

Unreal Engine

Herramienta de uso gratuito

Creado por Epic Games en 1998, emplea al mismo tiempo el lenguaje C++ y un sistema de *blueprints* que simplifica su uso **para aquellos que no dominan la programación.**

Es muy visual y sus últimas versiones han añadido la vista previa del renderizado para móviles y permiten acortar hasta en un 50 % los tiempos de compilación.

Unity 3D

Su funcionamiento es simple e intuitivo, así que es la alternativa preferida por quienes comienzan en este campo.

Como Unreal, permite utilizar un mismo script en diferentes plataformas (Mac, Windows, Linux…). Con este motor pueden crearse juegos tanto en 2D como en 3D o añadir elementos de ambos en un mismo proyecto.

RPG Maker

Permite a los usuarios crear sus propios videojuegos de rol.

La mayoría de las versiones incluyen un editor de mapas basado en conjuntos de fichas (los conjuntos de fichas se denominan chipsets en versiones anteriores a XP), un lenguaje de scripting sencillo (RGSS basado en Ruby, C++, Java para programación en smartphones o HTML5) para la programación de eventos y un editor de batallas.

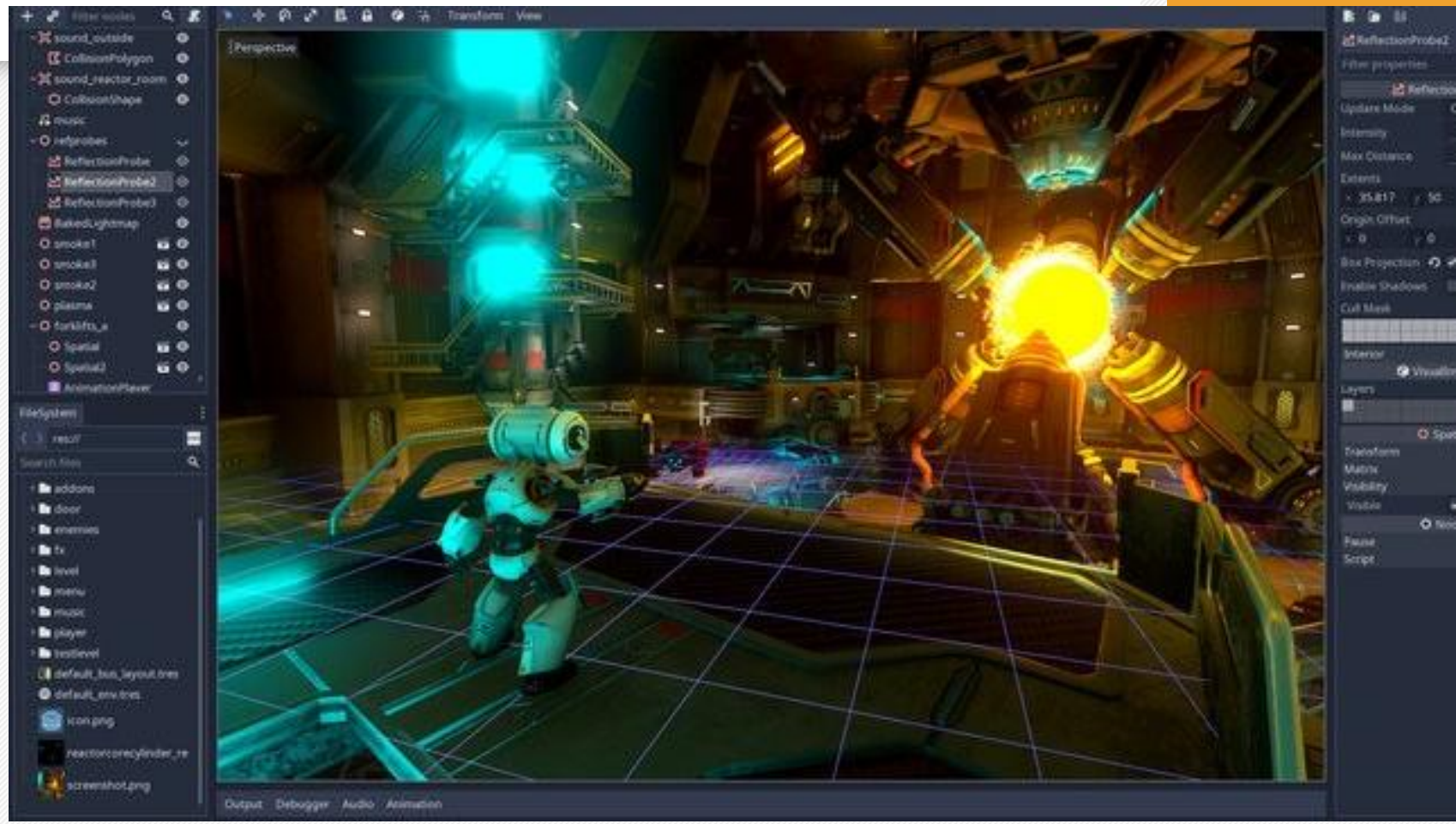
RPG Maker es uno de los programas clásicos más conocidos cuando se trata de crear juegos.

Godot Engine

Es totalmente **gratuito y de código abierto**, con licencia MIT.

Permite a los desarrolladores de videojuegos crear juegos 3D y 2D utilizando múltiples lenguajes de programación, como C++, C# y GDScript

Cuenta con una importante comunidad de usuarios, muy activa a la hora de corregir errores e incluir atractivas novedades.



Construct 3

Construct 3 es otro programa enfocado en juegos en 2D

Editor Basado en Navegador se ejecuta completamente en el navegador, lo que significa que no se requiere instalación.

En lugar de escribir código, Construct 3 utiliza un sistema de eventos visuales. Los usuarios crean eventos que consisten en condiciones y acciones, lo que simplifica la lógica del juego.

Scratch

Scratch es una plataforma de creación de videojuegos que enseña programación a niños.

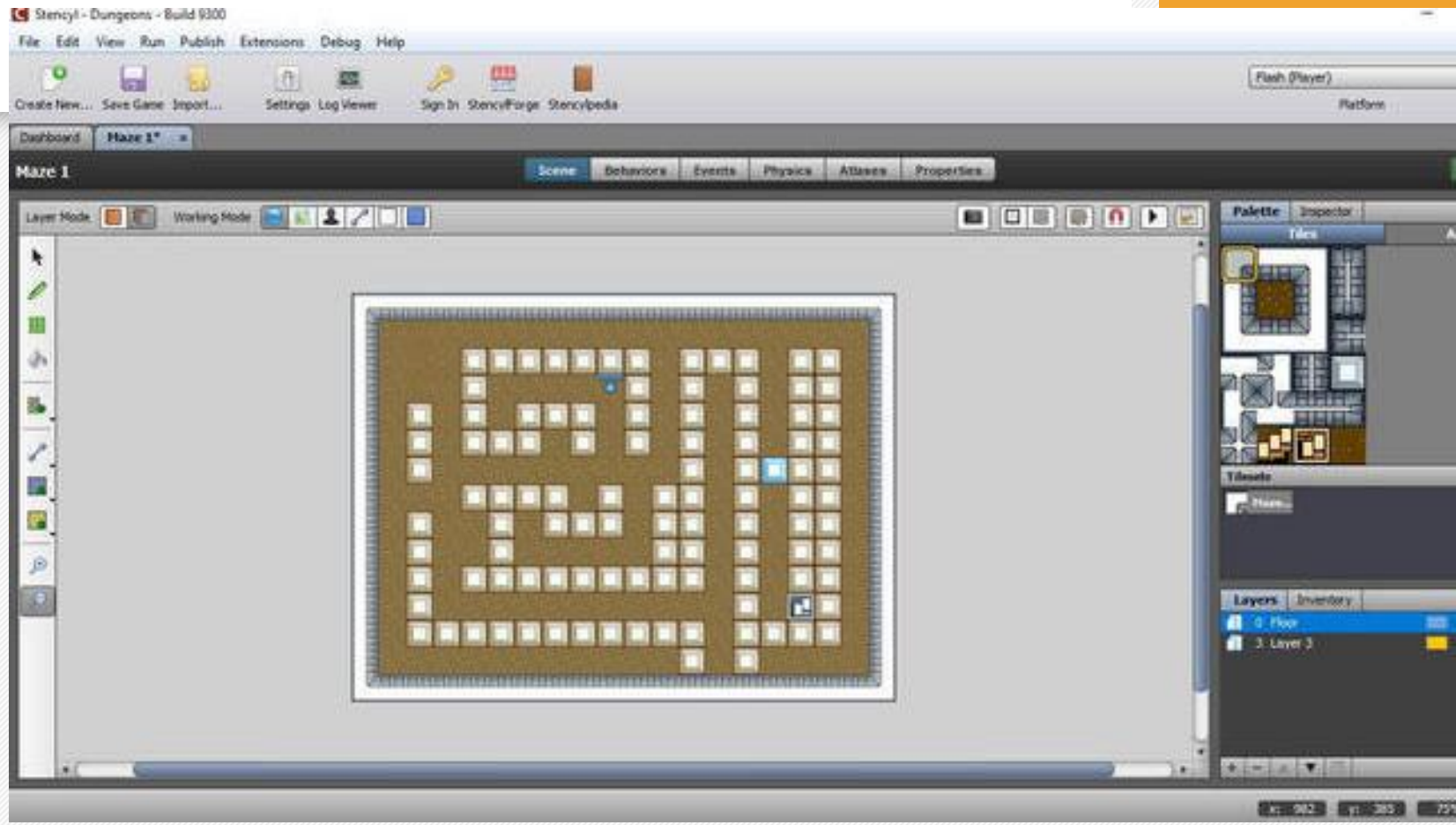
Scratch permite a los usuarios crear historias interactivas, juegos y animaciones mediante un sistema de bloques que se arrastran y sueltan

Stencyl

Stencyl es un término medio entre Scratch y Construct 3.

Es especialmente popular entre principiantes y educadores debido a su enfoque en el diseño visual y su facilidad de uso.

Escuelas y universidades lo emplean para enseñar los fundamentos del desarrollo de juegos y la lógica de programación.



GameMaker Studio 2

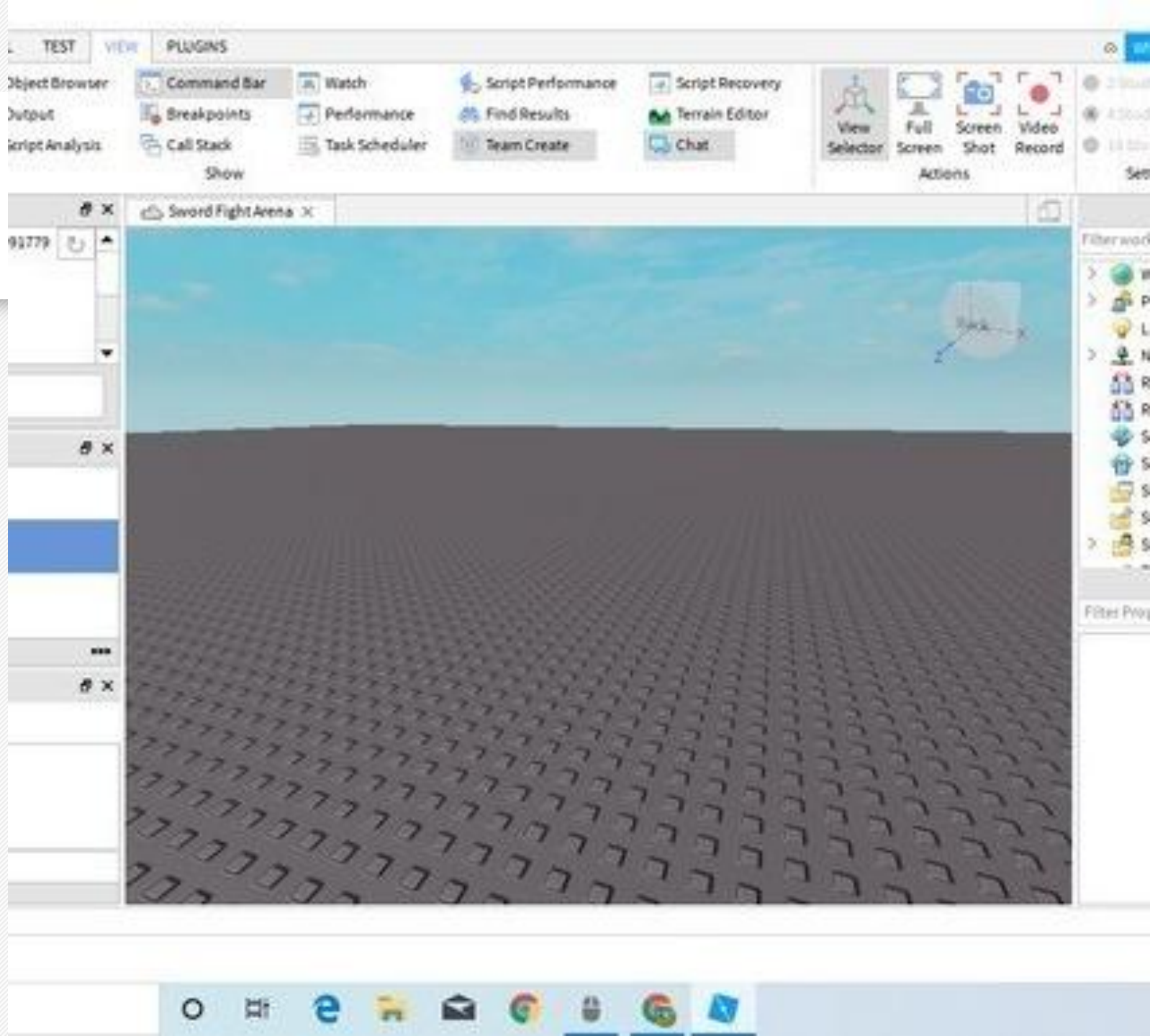
Recurre a su **propio sistema de programación**, permitiendo crear juegos multijugador o recurrir a la Inteligencia Artificial.

Como ocurre con Unity, su interfaz es muy sencilla y sus herramientas, fáciles de utilizar, ya que aplica el sistema *drag & drop* (arrastrar y soltar).

El lenguaje de scripting de GameMaker es: GML Code y GML Visual comúnmente comparado con JavaScript y lenguajes similares a C.

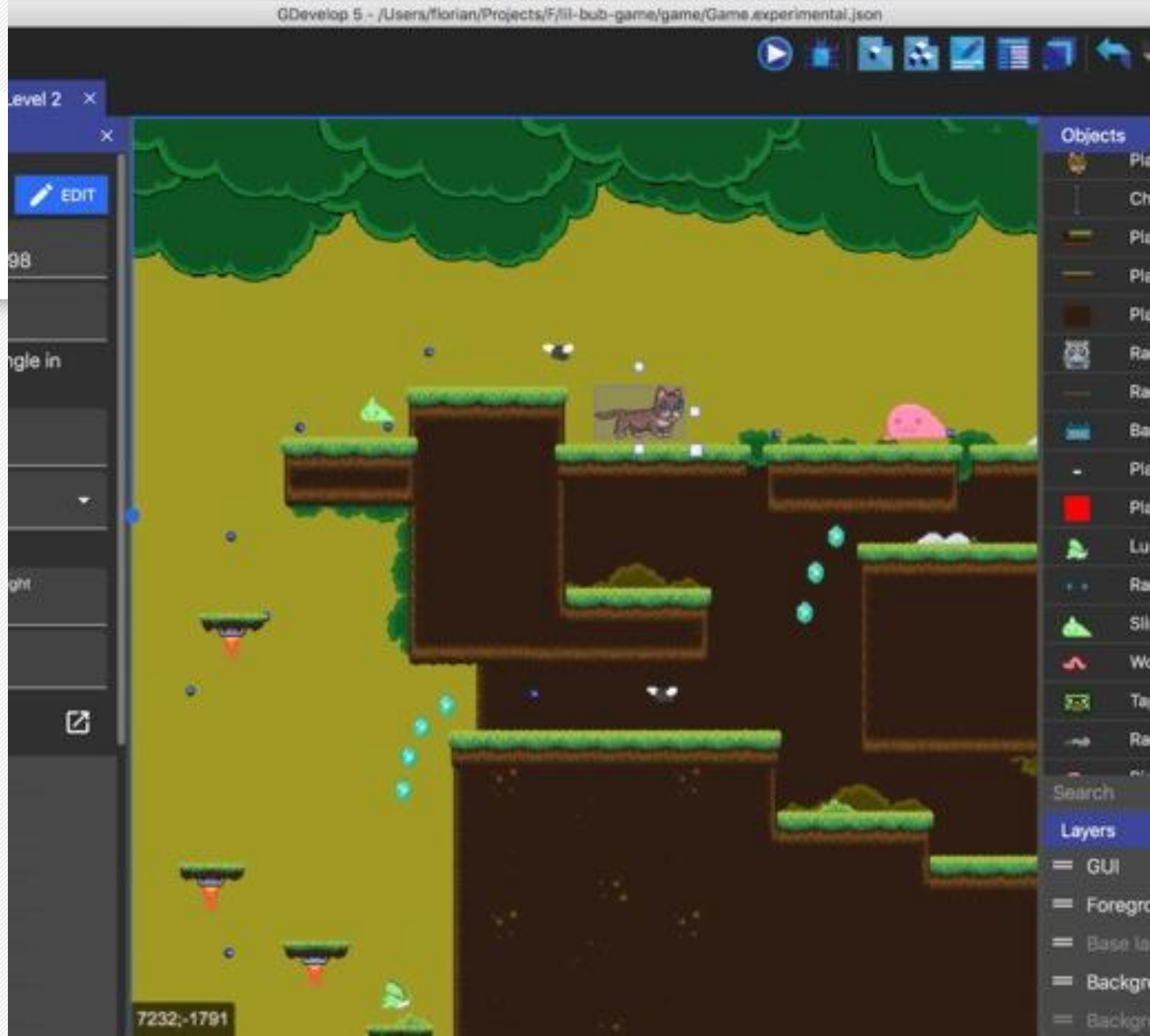
Roblox Studio

- Programa utilizado para crear juegos en Roblox
- Roblox Studio es el software utilizado para crear y diseñar los juegos que luego se pueden publicar en la plataforma Roblox.
- Entorno de Desarrollo Integrado (IDE)
- Los desarrolladores utilizan Lua, un lenguaje de scripting ligero y fácil de aprender, para programar la lógica del juego y crear interacciones complejas.
- ofrece una amplia gama de componentes y objetos prefabricados



GDevelop

- GDevelop es uno de los programas de creación de juegos que mayores compatibilidades posee para diferentes plataformas.
- Facilita la creación de juegos 2D sin necesidad de conocimientos avanzados de programación.
- Permite exportar juegos a múltiples plataformas, incluyendo HTML5, Android, iOS, Windows, Mac y Linux.



Adventure Game Studio

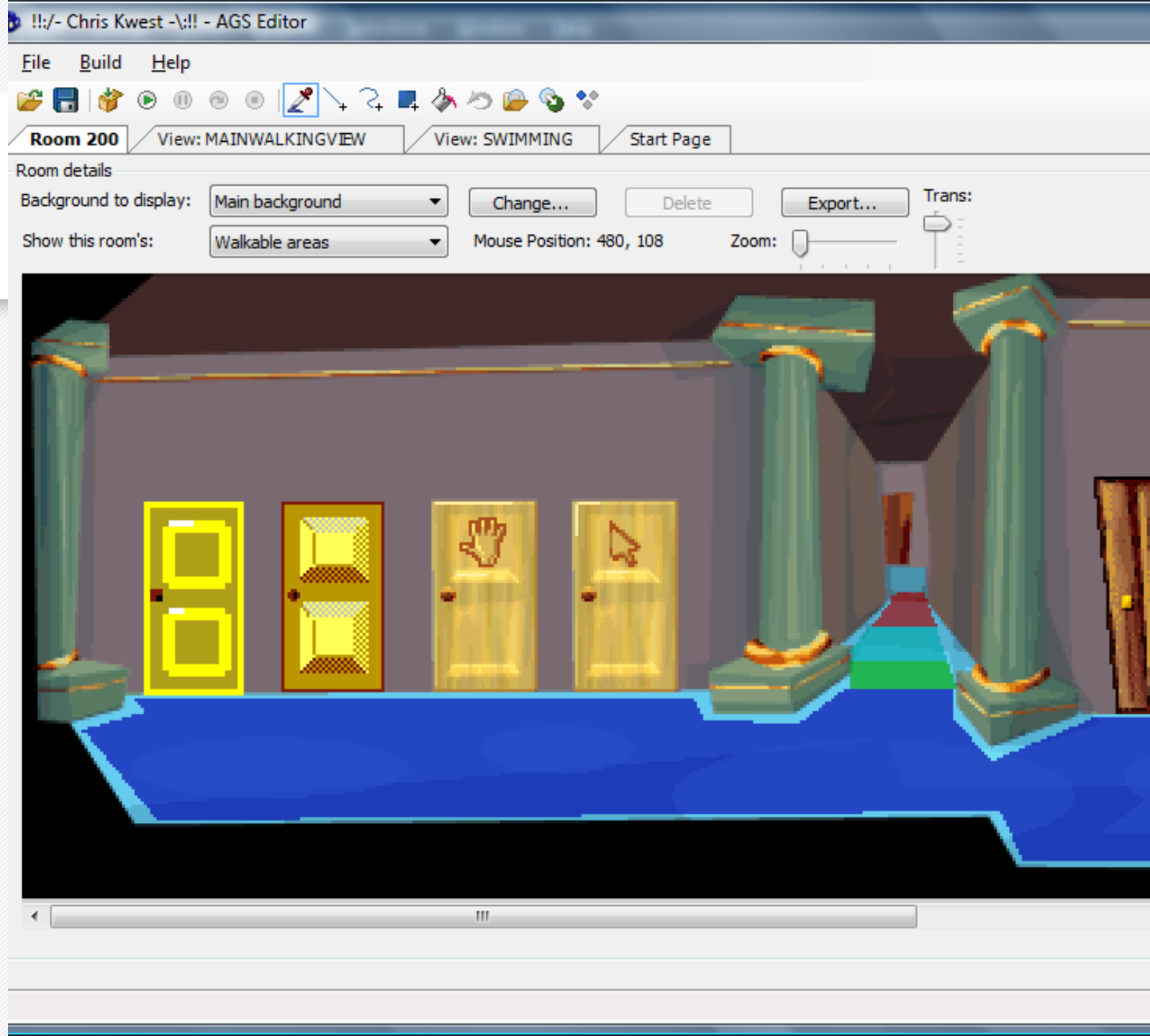
AGS Editor es un **IDE basado en Windows** para crear tus juegos de la forma más rápida y sencilla posible. Todo lo que necesitas, desde importar gráficos y escribir guiones de juegos hasta probar tu juego.

Especialización en Aventuras Gráficas.

Utiliza un lenguaje de scripting propio basado en C.

Posee editor integrado.

Soporta gráficos en alta resolución y una amplia gama de formatos de sonido.



¿ Preguntas ?